

Reforest

1. Propósito de la aplicación

ReForest ha sido creada para fomentar la reforestación responsable y la concienciación medioambiental mediante una plataforma digital que permite a los usuarios realizar donaciones destinadas a la plantación de árboles en ecosistemas específicos.

La aplicación ofrece un catálogo de especies arbóreas, proporcionando información detallada sobre cada una de ellas, como su zona geográfica, características principales y su impacto positivo en la captura de CO₂. De este modo, el usuario puede comprender la importancia de cada especie dentro de su ecosistema y realizar una donación informada.

Cada donación está vinculada a una única especie, siguiendo el principio de que cada ecosistema requiere árboles concretos para garantizar una reforestación sostenible y adecuada al entorno. Este enfoque evita la plantación indiscriminada y refuerza el compromiso con la conservación de la biodiversidad.

La solución se apoya en una API REST centralizada, consumida por una aplicación web. La aplicación web actúa como cliente principal, permitiendo la gestión completa de las donaciones.

Con esta iniciativa se busca no solo facilitar la participación ciudadana en acciones medioambientales, sino también sensibilizar sobre la importancia de la reforestación, contribuyendo así a un modelo de desarrollo más sostenible alineado con la Agenda 2030.

2. Relación con los ODS

La aplicación se encuentra alineada con la Agenda 2030 para el Desarrollo Sostenible, contribuyendo de forma directa a varios de los Objetivos de Desarrollo Sostenible (ODS) definidos por Naciones Unidas. En concreto, el proyecto se centra en los siguientes objetivos:

ODS 13 – Acción por el clima

La reforestación es una de las medidas más eficaces para combatir el cambio climático, ya que los árboles actúan como sumideros naturales de carbono, ayudando a reducir la concentración de CO₂ en la atmósfera.

La aplicación promueve la plantación de especies adecuadas a cada ecosistema, favoreciendo una reforestación eficiente y sostenible que contribuye a la mitigación del calentamiento global. Además, al informar sobre el impacto ambiental de cada especie, se fomenta la concienciación del usuario sobre la importancia de sus decisiones individuales en la lucha contra el cambio climático.

ODS 15 – Vida de ecosistemas terrestres

Este objetivo busca proteger, restaurar y promover el uso sostenible de los ecosistemas terrestres. La aplicación refuerza este compromiso al limitar cada donación a una única especie, garantizando que la plantación se adapte a las necesidades específicas de cada entorno natural.

De este modo, se evita la introducción de especies inadecuadas y se contribuye a la conservación de la biodiversidad, la recuperación de zonas degradadas y la protección de los hábitats naturales.

ODS 12 – Producción y consumo responsables

La plataforma fomenta un modelo de consumo responsable al ofrecer información clara y transparente sobre las especies disponibles, su impacto ambiental y el destino de las donaciones.

Este enfoque permite que los usuarios tomen decisiones conscientes y responsables, promoviendo prácticas sostenibles y una participación activa en iniciativas medioambientales con un impacto positivo a largo plazo.

3. Reflexión tecnológica y arquitectura

Stack tecnológico

Capa	Tecnología	Versión
API REST (Backend)	Spring Boot	3.2.2
Lenguaje	Java	21
Base de datos	MariaDB	11.3.2
Seguridad	Spring Security + JWT	0.12.6
Documentación API	SpringDoc OpenAPI / Swagger	2.3.0
Config server	Spring Boot + Spring Cloud Config Server	4.0.6 / 2025.1.1
Config client	Spring Cloud Config Client	2023.0.0
Entorno dev (API)	H2 in-memory	-
Frontend Web	React + TypeScript + Vite	19 / 7.x
HTTP Client (Web)	Axios	1.x
Router (Web)	React Router DOM	7.x
Contenedores	Docker Compose	-
CI/CD	GitHub Actions + Newman	-

Arquitectura y decisiones de diseño

La arquitectura cliente-servidor centraliza toda la lógica de negocio en la API REST. El cálculo del total de la donación ($\text{cantidad_arboles} \times \text{precio_plantacion}$) se realiza exclusivamente en el backend, evitando que los clientes envíen importes calculados y garantizando la integridad de los datos.

La restricción de una sola especie por donación simplifica el modelo de datos y elimina relaciones muchos-a-muchos. La configuración se gestiona de forma centralizada mediante un Spring Cloud Config Server independiente, con perfiles dev (H2) y prod (MariaDB). Los tres servicios —config server, base de datos y API— se orquestan con Docker Compose.

El desarrollo se realizó con Git Flow, con ramas de funcionalidad y revisión mediante Pull Requests, ejecutando automáticamente los tests de Newman en cada PR hacia main y develop.

4. Repositorios del proyecto

El proyecto se distribuye en tres repositorios bajo la organización GitHub ReForest-2DAM:

Repositorio	Contenido	Tecnología principal
ReForest	API REST (Backend), app web (Frontend), app Android, Docker Compose, Postman, CI/CD	Spring Boot 3.2.2 + React 19
ReForestConfigServer	Spring Cloud Config Server — sirve la configuración centralizada a la API	Spring Boot 4.0.6 + Spring Cloud 2025.1.1
ReForestConfigRepo	Ficheros .properties de configuración por perfil (dev y prod)	Properties (Git-backed)

5. Modelo de datos

Entidades

Especie (entidad maestra)

Atributo	Tipo	Descripción
id	int (PK)	Identificador único
nombre_comun	String	Nombre de la especie
descripcion	String	Descripción general
precio_plantacion	decimal	Coste de plantación por unidad
zona_geografica	String	Ecosistema o zona recomendada
co2_anual_kg	decimal	CO ₂ absorbido al año (kg)
altura_maxima_m	int	Altura máxima aproximada (m)
disponible	boolean	Disponible para donaciones
fecha_temporada	date	Temporada de plantación
image_url	String	URL de imagen de la especie

Usuario (entidad maestra)

Atributo	Tipo	Descripción
id	int (PK)	Identificador único
nombre	String	Nombre del usuario
email	String	Correo electrónico (único)
contrasena	String	Contraseña (hash BCrypt)
rol	String	Rol: USER / ADMIN

Donacion (entidad transaccional)

Atributo	Tipo	Descripción
id	int (PK)	Identificador único
fecha	date	Fecha de registro
nombre_donante	String	Nombre del donante
cantidad_arboles	int	Número de árboles donados
total_donado	decimal	Importe total (calculado en backend)
estado	String	Estado: PENDIENTE / COMPLETADA / CANCELADA
pagado	boolean	Si la donación está abonada
id_especie (FK)	int	Referencia a la especie
id_usuario (FK)	int	Referencia al usuario

Relaciones

- Especie 1:N Donación → una especie puede tener múltiples donaciones; cada donación pertenece a una única especie.
- Usuario 1:N Donación → un usuario puede realizar varias donaciones; cada donación pertenece a un único usuario.

6. API REST y especificación OpenAPI 3.0

La API REST desarrollada con Spring Boot centraliza el acceso a datos y la lógica de negocio.

Control de acceso

Ruta	Método	Acceso requerido
/auth/register, /auth/login	POST	Público
/especies, /especies/{id}	GET	Público
/especies/**	POST / PUT / PATCH / DELETE	Solo ADMIN
/donaciones/**	Todos	Autenticado (USER o ADMIN)
/usuarios/**	Todos	Solo ADMIN
/swagger-ui/**, /v3/api-docs/**	GET	Público

Endpoints de Especie (/especies)

Método	Ruta	Descripción	Respuesta
GET	/especies	Listado con filtros opcionales (disponible, zonaGeografica)	200
GET	/especies/{id}	Detalle de una especie	200 / 404
POST	/especies	Crear nueva especie	201 / 400
PUT	/especies/{id}	Modificación completa	200 / 404
PATCH	/especies/{id}	Modificación parcial de campos	200 / 404
DELETE	/especies/{id}	Eliminar especie	204 / 404

Endpoints de Donación (/donaciones)

Método	Ruta	Descripción	Respuesta
GET	/donaciones	Listado con filtros (estado, usuarioid)	200
GET	/donaciones/{id}	Detalle de una donación	200 / 404
POST	/donaciones	Crear donación (total calculado automáticamente)	201 / 400 / 404
PUT	/donaciones/{id}	Modificación completa	200 / 404
PATCH	/donaciones/{id}	Actualizar estado o pagado	200 / 404
DELETE	/donaciones/{id}	Borrado lógico → estado CANCELADA	204 / 404

Endpoints de Autenticación (/auth)

Método	Ruta	Descripción	Respuesta
POST	/auth/register	Registro de usuario nuevo (rol USER)	201
POST	/auth/login	Login → devuelve JWT + datos usuario	200 / 401

Códigos de estado HTTP

Código	Significado	Cuándo se usa
200 OK	Operación exitosa	Lecturas y modificaciones correctas
201 Created	Recurso creado	POST (especie, donación, registro)
204 No Content	Eliminación exitosa	DELETE correctos
400 Bad Request	Datos inválidos	Validación fallida (@Valid)
401 Unauthorized	No autenticado	Token ausente o inválido
403 Forbidden	Sin permisos	Rol insuficiente
404 Not Found	Recurso no encontrado	ID inexistente

7. Autenticación JWT

La API implementa autenticación stateless con JSON Web Tokens. El flujo completo:

- El usuario se registra en `/auth/register` o hace login en `/auth/login`.
- La API devuelve un JWT con email y rol en los claims. Expiración: 24 horas.
- El cliente incluye el token en cada petición: `Authorization: Bearer <token>`.
- El filtro `JwtAuthenticationFilter` valida el token en cada request.
- Spring Security aplica restricciones por rol (USER / ADMIN).

Las contraseñas se almacenan hasheadas con BCrypt. Los errores de autenticación devuelven JSON estructurado mediante `RestAuthEntryPoint` y `RestAccessDeniedHandler` personalizados.

El secreto JWT y otros parámetros se gestionan por el Config Server según el perfil activo:

Perfil	jwt.secret	jwt.expiration-ms
dev	dev-secret-key-reforest-very-long-for-testing-123456789	86400000 (24h)
prod	prod-secret-key-reforest-very-long-for-testing-123456789	86400000 (24h)

8. Spring Cloud Config

Config Server (ReForestConfigServer)

Repositorio independiente que actúa como servidor de configuración centralizado para la API. Detalles técnicos:

Aspecto	Valor
Spring Boot	4.0.6
Spring Cloud	2025.1.1
Puerto	8888
@EnableConfigServer	Sí
Backend de configuración	Git — repositorio ReForestConfigRepo (rama main)
URL del repositorio	https://github.com/ReForest-2DAM/ReForestConfigRepo

Config Repo (ReForestConfigRepo)

Repositorio Git que almacena los ficheros .properties por perfil. El Config Server los sirve automáticamente según el perfil solicitado:

Fichero	Perfil	Base de datos	Uso
ReForest-dev.properties	dev	H2 in-memory (jdbc:h2:mem:reforestdb)	Desarrollo local y CI/CD
ReForest-prod.properties	prod	MariaDB (jdbc:mariadb://reforest-db:3306/reforest)	Producción / Docker

Configuración relevante por perfil

Perfil dev (H2)

- Driver: org.h2.Driver — base de datos en memoria, se crea y destruye en cada arranque.
- spring.jpa.hibernate.ddl-auto=create-drop — el esquema se genera automáticamente.
- Consola H2 habilitada en /h2-console.
- Nomenclatura JSON: SNAKE_CASE (spring.jackson.property-naming-strategy).

Perfil prod (MariaDB)

- Driver: org.mariadb.jdbc.Driver — conecta al contenedor reforest-db:3306.
- spring.jpa.hibernate.ddl-auto=update — preserva los datos entre reinicios.
- Dialecto Hibernate: org.hibernate.dialect.MariaDBDialect.

Admin fijo

Ambos perfiles definen un usuario administrador fijo que se crea al arrancar la aplicación:

- Email: admin@gmail.com
- Password: admin123

9. Docker y despliegue

El Docker Compose del repositorio principal orquesta tres servicios:

Servicio	Imagen / Build	Puerto	Descripción
reforest-db	mariadb:11.3.2	3307:3306	MariaDB con healthcheck (innodb_initialized)
config-server	Build ReForestConfigServer/	8888:8888	Spring Cloud Config Server
reforest-api	Build Backend/	8080:8080	API REST con perfil prod; espera a DB y config

La API espera a que la base de datos esté healthy y a que el config server esté levantado antes de arrancar, garantizando el orden correcto de inicialización. El config server se construye desde su propio repositorio (ReForestConfigServer) con su Dockerfile multi-stage (Maven build + JRE 21).

Dockerfile del Config Server

El Config Server usa una imagen multi-stage para minimizar el tamaño final:

- Etapa build: maven:3.9.9-eclipse-temurin-21 — compila omitiendo tests.
- Etapa runtime: eclipse-temurin:21-jre — solo el JRE y el JAR resultante.
- Puerto expuesto: 8888.

10. Testing (API)

Tests automatizados con Postman + Newman

Se ha configurado una colección Postman completa para las entidades Especie y Donación, cubriendo:

- Respuestas 200 / 201 / 204 para operaciones exitosas.
- Respuestas 400 para datos inválidos.
- Respuestas 404 cuando el recurso no existe.
- Validación del código de estado y la estructura de la respuesta.

Tests unitarios con JUnit 5

El backend incluye tests con Spring Boot Test:

- JwtServiceTest — generación y validación de tokens.
- AuthFlowTest — flujo completo de registro y login.
- AccessControlTest — verificación de protección de rutas por rol.
- UserPersistenceTest — persistencia de usuarios en base de datos.
- UsuarioSerializationTest — serialización/deserialización del modelo.

GitHub Actions CI/CD

El workflow se dispara automáticamente en cada Pull Request contra main y develop:

- Hace checkout del repositorio principal y del ReForestConfigServer.
- Levanta el config server con mvn spring-boot:run (espera 25 segundos).
- Arranca la API Spring Boot en modo dev/H2 (espera 35 segundos).
- Instala Newman y ejecuta la colección Postman completa.

11. Aplicación Web (Frontend)

La aplicación web está desarrollada con React 19 + TypeScript + Vite y actúa como el cliente principal de la API.

Páginas y funcionalidades

Ruta	Componente	Acceso	Funcionalidad
/	Home	Público	Página de inicio con resumen de la app
/login	Login	Público	Autenticación JWT, persistencia en localStorage
/register	Register	Público	Registro de nuevo usuario
/especies	EspeciesList	Autenticado	Catálogo con filtros; crear/editar/eliminar (ADMIN)
/especies/:id	EspecieDetail	Autenticado	Detalle + botón Plantar Ahora
/donaciones	DonacionesList	Autenticado	Gestión de donaciones según rol
/dashboard	Dashboard	Autenticado	Métricas, filtros y tabla ordenable por rol
/usuarios	UsuariosList	Solo ADMIN	Gestión de usuarios del sistema

Dashboard diferenciado por rol

- ADMIN — ve todas las donaciones y métricas globales (árboles, recaudación, usuarios, canceladas). Puede filtrar por estado, fechas y buscar por texto.

Dashboard — Administración

Resumen global del sistema

10 Donaciones	3097.50€ Total recaudado	267 Árboles plantados	5340 kg CO ₂ compensado (est.)	5 Usuarios registrados	7 Especies en catálogo
------------------	-----------------------------	--------------------------	--	---------------------------	---------------------------

Buscar donante
Nombre del donante...

Estado
Todos

Desde
dd/mm/aaaa

Hasta
dd/mm/aaaa

Limpiar

Mostrando 10 de 10 donaciones — 267 árboles · 3097.50€ · 5340 kg CO₂

Fecha ↓	Donante	Árboles ↑	Total ↑	Estado ↓	Pagado	Usuario ID	Especie ID
22 jun 2025	Ana	12	120.00€	Pendiente	X No	#5	#2
10 jun 2025	Pepe	60	750.00€	Completado	✓ Sí	#4	#1
15 may 2025	Cristina	40	600.00€	Completado	✓ Sí	#3	#8
02 may 2025	Andrea	25	187.50€	Pendiente	X No	#2	#7
18 abr 2025	David	15	195.00€	Completado	✓ Sí	#1	#6

- USER — ve únicamente sus propias donaciones con métricas personales.

Dashboard — Pepe

Resumen de tus donaciones

2 Donaciones	1035.00€ Total recaudado	90 Árboles plantados	1800 kg CO ₂ compensado (est.)
-----------------	-----------------------------	-------------------------	--

Buscar donante

Estado

Todos

Desde

Hasta

Limpiar

Mostrando 2 de 2 donaciones — 90 árboles · 1035.00€ · 1800 kg CO₂

Fecha ↑	Donante	Árboles ↑	Total ↑	Estado ↑	Pagado
10 jun 2025	Pepe	60	750.00€	Completado	✓ Sí
20 mar 2025	Pepe	30	285.00€	Completado	✓ Sí
Total (2)		90	1035.00€		

La tabla del dashboard permite ordenación reactiva por fecha, total donado, cantidad de árboles y estado.

Internacionalización

La aplicación soporta español e inglés mediante LanguageContext (React Context API). El cambio de idioma es en tiempo real desde la navegación, sin recarga de página.

12. Diseño de interfaces y Leyes de Gestalt

Ley de Proximidad

En el listado de especies, cada card agrupa los elementos relacionados: nombre, zona, precio, disponibilidad y acción de Plantar Ahora. Esta agrupación reduce la carga cognitiva al navegar por el catálogo.

Ley de Similitud

Todas las cards comparten el mismo diseño (estructura, tipografías, distribución). Los botones mantienen estilos consistentes para acciones equivalentes. La jerarquía visual repetida permite que el usuario aprenda el patrón y escanee la información rápidamente.

13. Colección Postman

El repositorio incluye la colección completa (postman/ReForest.postman_collection.json) con:

- Entorno local — apunta a <http://localhost:8080> para pruebas en desarrollo.
- Entorno de CI — configurado para el workflow de GitHub Actions (H2).
- Cobertura completa — todas las operaciones CRUD de Especie y Donación, con casos 2xx, 400 y 404.

14. Anexo IA

Las herramientas de IA generativa se han utilizado a lo largo del proyecto como apoyo en la resolución de errores y como punto de partida para soluciones técnicas. En todos los casos, el código generado fue revisado, adaptado y comprendido por los integrantes del equipo antes de su integración.

Junto a esta documentación se adjuntan los ficheros .md con las conversaciones relevantes de cada integrante del grupo.